Ejercicio 18. Dado un arreglo de pares <estudiante, nota>, queremos devolver un arreglo de pares <estudiante,promedio>, ordenado en forma descendente por promedio y, en caso de empate, por número de libreta.

Los estudiantes se representan con un número de libreta (puede considerar que es un número de a lo sumo 10 dígitos), y la nota es un número entre 1 y 100. La cantidad de estudiantes se considera no acotada.

Se pide dar un algoritmo que resuelva el problema con complejidad O(n + m log m), donde n es la cantidad total de notas (es decir, el tamaño del arreglo de entrada), y m es la cantidad total de estudiantes. Ejemplo:

Entrada: <12395, 72>, <45615, 81>, <12395, 94>, <45615, 100>, <78920, 81>, <12395, 90>, <45615, 71>, <78920, 50>
Salida: <12395, 85.3333>, <45615, 84>, <78920, 65.5>

a) Describa cada etapa del algoritmo con sus palabras, y justifique por qué cumple con las complejidades pedidas.

b) Escriba el algoritmo en pseudocódigo. Puede utilizar (sin reescribir) todos los algoritmos y estructuras de datos vistos en clase

```
function OrdenarXPromedio(in A: array<(int,int)>): array<(int,float)>{
   var lista: lista<(int,float)> := new ListaEnlazada<(int,float)>()
   var res: array<(int,float)> := []

   RadixSort(A, 10)     -- Ordena por 1er componente de forma descendente
                        -- (Las libretas son acotadas a 10 dígitos)
                        -- Complejidad: O(10*n) = O(n)
   if |A| > 0
       var libretaActual: int := A[0][0]                          --O(1)
       var sumaNotas: int     := A[0][1]                          --O(1)
       var actual: int    := 1                                    --O(1)
       var cantNotas: int := 1                                    --O(1)
       var promedio: float := 0.0                                 --O(1)

       -- Recorremos el arreglo y vamos acumulando la suma de las libretas iguales
       -- para luego promediarlas. Sabemos que están juntas porque acabamos de ordenarlas
       while actual < |A|                                         --O(n)
           if libretaActual == A[actual][0]                           --O(1)
               sumaNotas := sumaNotas + A[actual][1]                  --O(1)
               cantNotas := cantNotas + 1                             --O(1)
           else
               promedio := sumaNotas/cantNotas                        --O(1)
               agregarAtras(lista, tupla(libretaActual, promedio)) --O(1)

               libretaActual := A[actual][0]                          --O(1)
               sumaNotas  := A[actual][1]                             --O(1)
               cantNotas := 1                                         --O(1)
           end if
           actual := actual + 1                                       --O(1)
       end while
       -- Agregamos el último estudiante
       promedio := sumaNotas/cantNotas                            --O(1)
       agregarAtras(lista, tupla(libretaActual, promedio))        --O(1)

       res := listToArray(lista)                                  --O(m)
       MergeSort(res, ordenaPor2daComp_deMayorAMenor)
               -- En caso de empate, radix ya había ordenado por libreta.
               -- Y como mergeSort es ESTABLE, se mantiene el orden previo.
               -- Complejidad: O(m*log(m))
   end if
   return res
}
-- COMPLEJIDAD TOTAL: O(n)  +  O(n)  +   O(m)   +  O(m*log(m)) =
--                  (Radix)  (while)  (list2Array) (mergeSort)
--                   = O(n + m + m*log m)
--                   = O(n + m*log m)
```

```
-------------------------------------------------------------------------------------
-- SOLUCIÓN ALTERNATIVA CON TRIE:
-- Aprovechamos que las libretas están acotadas por 10 digitos
-------------------------------------------------------------------------------------
function OrdenarXPromedioTrie(in A: array<(int,int)>): array<(int,float)>{

   var notasPorEstudiante: DictTrie<int, (int,int)> := new DictTrie<int, (int,int)>()
   --    El tipo del dicc es : <libreta, (sumaNotas, cant)>

   -- Acumulamos suma y cantidad de notas por libreta en el dicc
   for i = 0; i < |A|; i++                                                  --O(n)
       var libreta: int := A[i][0]                                          --O(1)
       var nota: int    := A[i][1]                                          --O(1)

       if está(notasPorEstudiante, libreta)                                 --O(1)*
           var par: (int,int) := obtener(notasPorEstudiante, libreta)       --O(1)*
           par[0] := par[0] + nota                                          --O(1)
           par[1] := par[1] + 1                                             --O(1)
           definir(notasPorEstudiante, libreta, par)                        --O(1)*
       else
           definir(notasPorEstudiante, libreta, tupla(nota, 1))             --O(1)*
       end if
   end for

   -- Recorremos los m estudiantes del Trie para calcular su promedio
   var lista: ListaEnlazada<(int,float)> := new ListaEnlazada<(int,float)>();
   for each libreta in notasPorEstudiante:                                  --O(m)
       var par: (int,int)  := obtener(notasPorEstudiante, libreta)          --O(1)*
       var promedio: float := par[0] / par[1]                               --O(1)
       agregarAtras(lista, tupla(libreta, promedio))                        --O(1)
   end for

   var res: array<(int,float)> := listToArray(lista)                -- O(m)
   RadixSort(res, 10) -- Ordena por 1er componente de forma descendente
                      -- Complejidad: O(10*m) = O(m)
   MergeSort(res, ordenaPor2daComp_deMayorAMenor) -- Complejidad: O(m log m)
   return res
}
-- OBSERVACION: las complejidades anotadas como O(1)* vienen de que el tamaño
--              maximo de las claves del DiccTrie es 10 => O(10) = O(1).

-- COMPLEJIDAD TOTAL: O(n)  +  O(m)   +    O(m)    +  O(m log m)  =
--                  (for)  (for each) (listToArray   (mergeSort)
--                                     y RadixSort)
--                  = O(n + m log m)
```

Ejercicio de Parcial. Se está por realizar un gran censo de perros en la Ciudad de Buenos Aires. La información recaudada llegará en forma de un arreglo de tuplas <Nombre, Raza, Edad, Peso>. Las razas son *strings* de largo acotado *L*, pero no sabemos cuántas razas pueden existir, aunque sí sabemos que entre ellas está la mejor raza de todas: raza Perro. Las edades serán enteros positivos entre 1 y 20 y los pesos serán de tipo float.

Proponga un algoritmo que organice la información en dos arreglos:

- Uno con los perros raza Perro ordenados por edad y, en caso de empate, por peso, ambos en forma ascendente
- Otro arreglo con la información de las razas y la cantidad de perros de cada una en forma de tuplas <Raza, int>, ordenadas en forma decreciente por la cantidad y, en caso de empate, alfabéticamente por la raza

La complejidad temporal del algoritmo debe ser O(n + p*log(p)), donde *n* es el total de perros y *p* el total de perros raza Perro.
Ejemplo Si recibimos:
[<Nulus, Perro, 4, 15.3>, <Joe, Cocker, 14, 8.0>, <Lady, Caniche, 10, 6.1>, <Dobby, Perro, 7, 18.3>, <Res, Dogo, 3, 50.0>, <Haru, Perro, 7, 30.0>, <Clementina, Perro, 14, 10.0>, <Choqui, Caniche, 16, 5.0>]

Deberíamos devolver:
P: [<Nulus, 4, 15.3>, <Dobby, 7, 18.3>, <Haru, 7, 30.0>, <Clementina, 14, 10.0>]
R: [<Caniche, 2>, <Cocker, 1>, <Dogo, 1>]

```
--                                  nombre, raza, edad, peso
function CensoDePerros(in A: array<(string,string,int,float)> :
--                                 nombre, edad, peso          raza, cantidad
                              (array<(string,int,float)> , array<(string,int)> ) {
   var listaP: ListaEnlazada<(string,int,float)> := new ListaEnlazada<(string,int,float)>();
   var listaR: ListaEnlazada<string> := new ListaEnlazada<string>();

   -- 1. Separamos los perros de raza "Perro" del resto
   for i = 0; i < |A|; i++                                                --O(n)
       if A[i][1] == "Perro"                                                  --O(L) = O(1)
           agregarAtras(listaP, tupla(A[i][0], A[i][2], A[i][3]))             --O(1)
       else
           agregarAtras(listaR, A[i][1])                                      --O(1)
       end if
   end for

   -- 2. Armamos  P (perros de raza "Perro")
   var P: array<(string,int,float)> := listToArray(listaP)                --O(p)
   MergeSort(P, menorAMayor_Por3raComp)
                  -- Ordena por peso (float no acotado)
                  -- Complejidad: O(p*log(p))
   BucketSort(P, 20, h)
                  -- La función h ordena por 2da componente (edad: [1..20]) de menor a mayor
                  -- Ejemplo: h((nombre,edad,peso)) = edad - 1
                  -- Como BucketSort es estable, en caso de empate mantiene el orden anterior
                  -- Complejidad: O(p + 20) = O(p)

   -- 3. Armamos R (resto de las razas)
   var razas: array<string> := listToArray(listaR)                        --O(n - p) = O(n)
   RadixSort(razas, L)     --(*)
                  -- Ordena lexicográficamente las razas
                  -- Complejidad: O(L * n) = O(n)

   var conteoRazas: ListaEnlazada<(string,int)> := new ListaEnlazada<(string,int)>();
   if |razas| > 0
       var razaActual: string := razas[0]                                 --O(1)
       var cantPerros: int    := 1                                        --O(1)
       var actual: int        := 1                                        --O(1)

       while actual < |razas|                                             --O(n)
           if razaActual == razas[actual]                                     --O(L) = O(1)
               cantPerros := cantPerros + 1                                   --O(1)
           else
               agregarAtras(conteoRazas, tupla(razaActual, cantPerros))       --O(1)
               razaActual := razas[actual]                                    --O(1)
               cantPerros := 1                                                --O(1)
           end if
           actual := actual + 1                                               --O(1)
       end while
       -- Agregamos la última raza
       agregarAtras(conteoRazas, tupla(razaActual, cantPerros))           --O(1)
   end if

   var R: array<(string,int)> := listToArray(conteoRazas)             --O(n)
   BucketSort(R, |A|, mayorAMenor_Por2daComp)
         -- La cantidad de perros por raza está acotada entre 1 y n (|A|)
         -- En caso de empate, RadixSort ya había ordenado alfabéticamente por 1ra comp (raza)
         -- Complejidad: O(|R| + |A|) = O(n + n) = O(n)  (porque |R| ≤ n)

   return tupla(P, R)
}
-- COMPLEJIDAD TOTAL: O(n) + O(p*log p) +    O(p)    + O(n)  + O(n)  +   O(n) =
--                   (for)   (MergeSort) (BucketSort) (Radix) (while) (bucketSort)
--                   = O(n + p + p*log p) = O(n + p*log p)
-- OBSERVACION (*):
--  Acá en la clase del Turno Mañana dije al final de la clase que se podría hacer un
--  BucketSort(razas, cantRazas, ordenLexicografico), pero ESTABA MAL porque me imagine
--  al algoritmo armando buckets para cada raza, y que solo las cuente (como CountingSort)
--  pero me olvide de la parte de ordenar jajaja. Y acá la idea era ordenar lexicograficamente
--  por lo que lo ideal era usar RadixSort.

-- 2da OBSERVACION: Tambien se podría usar un diccTrie para las razas que no son "Perro", dado
--  que los nombres de las razas están acotados por L. La complejidad queda igual.
```

Ejercicio 12. Se desea ordenar los datos generados por un sensor industrial que monitorea la presencia de una sustancia en un proceso químico. Cada una de estas mediciones es un número entero positivo. Dada la naturaleza del proceso se sabe que, dada una secuencia de n mediciones, a lo sumo $\lfloor \sqrt{n} \rfloor$ valores están fuera del rango [20, 40].

Proponer un algoritmo O(n) que permita ordenar ascendentemente una secuencia de mediciones y justificar la complejidad del algoritmo propuesto.

```
function OrdenarMediciones(in A: array<int>): array<int>{
   var enRango: ListaEnlazada<int> := new ListaEnlazada()      --O(1)
   var fueraDeRango: ListaEnlazada<int> := new ListaEnlazada() --O(1)
   var res: array<int> := new array<int>(length(A))            --O(n)

   for i = 0; i < |A|; i++                    --O(n)
       if A[i] >= 20 && A[i] <= 40 then
           agregarAtras(enRango,A[i])             --O(1)
       else
           agregarAtras(fueraDeRango,A[i])        --O(1)
       end if
   end for

   var cantidadAcotada: array<int> := listToArray(fueraDeRango) --O(√n)
   var rangoAcotado: array<int> := listToArray(enRango)         --O(n)
   -- Nota: El TAMAÑO de la lista "enRango" está en el rango [n - √n, n]
   -- por lo que la complejidad de recorrerlo es O(n) en el peor caso.

   SelectionSort(cantidadAcotada, deMenorAMayor)
       -- La cantidad de estos elementos es a lo sumo √n
       -- Complejidad: O(√n * √n) = O(√n²) = O(n)
       -- OBS: Se puede usar cualquier algoritmo de ordenamiento por comparación
   BucketSort(rangoAcotado, 21, deMenorAMayor_enRango20a40)
       -- El rango de valores está entre 20 y 40, por lo que necesito 21 buckets
       -- La función de mapeo debe organizar bien los valores (Ej: h(x) = x - 20).
       -- Complejidad: O(n + 21) = O(n)

   -- Unimos las dos listas de forma ordenada.
   res = Merge(cantidadAcotada, rangoAcotado)    --O(n)
   return res
}
-- COMPLEJIDAD TOTAL: O(n)  +  2*O(n)   +   O(n)   +   O(n)   + O(n)   = O(n)
--                   (for) (list2Array) (Selection) (Bucket) (merge)
```

DISCLAIMER de resolución (Luis):

Esta es la forma que resolvimos los ejercicios en el TM, donde trate de estandarizar la sintaxis para que no hayan muchas dudas al respecto de la notación.
Esto NO significa que ustedes tengan que escribir con exactamente la misma sintaxis. Pueden usar notación Python, C++, Java, etc. Los ejercicios piden que se resuelva en **pseudocodigo**, por lo que uno puede usar la sintaxis que le sea más comoda, mientras **se entiendan las operaciones realizadas**. Algo fundamental es que la sintaxis que usen NO modifique drasticamente la complejidad de las operaciones que usen.

Lo más importante es que usen adecuadamente las funciones vistas en la Clase Práctica de Sorting, y el Apunte de Tads y Modulos del Campus, con las complejidades correctas, y justificadas en donde sea necesario.
Todas los comentarios que hice en la resolución fueron con el objetivo de aclarar dudas de lo que escribí en el código, pero a la vez son ejemplo de como se puede justificar en la resolución de su parcial.

Por último, **no hay una sola forma de resolver** esta clase de ejercicios, ni hay solo una función de ordenamiento que los resuelva. Las resoluciones van a variar en función de como lo pensaron ustedes. Por esa razon la justificación de sus decisiones es esencial al momento de resolver los ejercicios.